Université Gustave Eiffel

A First Analysis of the FDR String Matcher

We extract a pseudocode of a simplified version of FDR for later analysis. A Python implementation so that we can inject instruction counter is also given in the repository. We begin with the pseudocode. Then we solve special cases and try to formulate the problem in a more general way.

1 Pseudocode

The algorithm extends Shift-Or algorithm for multiple patterns. The conventions are as follows. The number of bits of a character is 8. There are 8 buckets $B_0, \ldots, B_7$. Each register is of size 128 bits. We access the bit indices in a register and the character positions in a string *right to left*.

The compilation phase takes a list of patterns, put them into buckets and builds a mask for each character. The mask for a character c is a bitmask of size 128 where the left-most 64 bits are set to 0. In the right-most 64 bits, the $(8i + j)$-th bit $(0 \leq i, j \leq 7)$ is set to 0 if either

(i) c appears in position i (indexing from the right) of a pattern in bucket B_j, or

(ii) bucket B_j is not empty and patterns in B_j have length less than or equal to i. In this case it is called a *padding bit*.

Otherwise, the bit is set to 1.

For example, consider the buckets $[[ab, bc], [abc], [], [], [], [], [], []]$. Then the mask for each character is as follows. The padding bits are in bold.

$$M[a] = \underbrace{00\ldots00}_{64}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 11111\mathbf{10}\ 11111110\ 11111111$$

$$M[b] = \underbrace{00\ldots00}_{64}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 11111\mathbf{1}0\ 11111100\ 11111110$$

$$M[c] = \underbrace{00\ldots00}_{64}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 11111\mathbf{1}0\ 11111111\ 11111101$$

$$M[x] = \underbrace{00\ldots00}_{64}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 1111\mathbf{00}\ 11111\mathbf{1}0\ 11111111\ 11111111.$$

In the masks above, x is any character rather than a, b, c.

The execution phase takes a text and the data structure built in the compilation phase. It maintains a state mask R. It is better to interpret the bits in R as follows. Each iteration processes 8 bytes. At iteration `ITER`, the $(8i + j)$ bit $(0 \leq i \leq 15, 0 \leq j \leq 7)$ is 0 means that there is currently *no contradiction* that there is a match of a pattern in bucket B_j ending at position $8 \times \texttt{ITER} + i$. If bucket B_j has patterns of length ℓ_j, then there cannot be a match in this bucket at position less than $\ell_j - 1$. Therefore, we initialize the state mask to all 0's, except for $8i + j$ where $0 \leq i < \ell_j - 1$. After an iteration, we update $R := R \gg 64$, continuing matching the next 8 bytes. By that way, we ensure that at each position, the prefix of length at most 8 is considered.

Coming back to the example, the initial state mask is

$$R = \underbrace{00\ldots00}_{104}\ 11111100\ 11111110\ 11111111.$$

Let us execution on the text *babc*. Since there are only 4 character in the text, there is only one iteration. We give details about scanning each character as follows, where we consider the right-most 4 bytes. The newly bit set to 1 after each character is scanned is underlined.

Scan	Update	Result
b	$R \leftarrow R \mid (M[b] \ll 0)$	... 11111100 111111$\underline{\mathbf{1}}$0 11111110 11111111
a	$R \leftarrow R \mid (M[a] \ll 1)$	... 11111100 11111110 1111111$\underline{\mathbf{1}}$ 11111111
b	$R \leftarrow R \mid (M[b] \ll 2)$	... 11111100 11111110 11111110 11111111
c	$R \leftarrow R \mid (M[c] \ll 3)$	... 1111110$\underline{\mathbf{1}}$ 11111110 11111110 11111111

Now we report that there is a match at bucket B_0 that ends at position 1, a match at bucket B_1 that ends at position 2 and a match at bucket B_0 that ends at position 3. Since we already know the length of the patterns in each bucket, we can retrieve the beginning position of those potential matches and verify them by exact string comparison. In this case, those three potential matches are indeed matches. We give the pseudocode as in Algorithm 1 and Algorithm 2.

Algorithm 1 Compilation phase of the FDR string matcher

```
1: Assign each pattern to a bucket B_1, ..., B_b.
2: M ← a dictionary mapping each character to a mask.
3: for c = 00000000 to 11111111 do
4:     M[c] ← 00...00 11...11 (64 zeros, 64 ones)          ▷ Initialize.
5:     for j = 0 to 7 and B_j is not empty do
6:         ℓ_j ← length of the longest pattern in bucket B_j.
7:         for i = ℓ_j to 7 do
8:             M[c][8i + j] ← 0                            ▷ Set padding bits to 0.
9:         endfor
10:    endfor
11: endfor
12: for j = 0 to 7 do
13:    for each pattern x in bucket B_j do
14:        for i = 0 to length(x) − 1 do
15:            M[x[i]][8i + j] ← 0
16:        endfor
17:    endfor
18: endfor
```

Algorithm 2 Execution phase of the FDR string matcher

1: $R \leftarrow$ a bitmask of size 128 where the $(8i + j)$-th bit is 0 if $i \geq \ell_j - 1$ and 1 otherwise.
2: $T \leftarrow$ the text to be scanned.
3: **for** $\texttt{ITER} = 0$ to $\lfloor \frac{\text{length}(T)}{8} \rfloor - 1$ **do**
4: $\quad V \leftarrow T[8 \times \texttt{ITER} \ldots 8 \times \texttt{ITER} + 7]$
5: $\quad$ **for** $j = 0$ to 7 **do**
6: $\quad\quad R \leftarrow R|(M[V[j]] \ll j)$
7: $\quad$ **endfor**
8: $\quad$ Report potential matches at positions where the corresponding bits in R are 0. Verify those potential matches by exact string comparison.
9: $\quad R \leftarrow R \gg 64$
10: **endfor**

Coming back to the buckets in the example. The current algorithm cannot give contradiction to the text *ac* at position 1 for bucket B_0. *Super-characters* are introduced. Let s be the number of bits in a super-character. For example, instead of assigning $M[a]$ as in the first step of the compilation phase, we build a mask for the super-character

$$(b[0 \ldots s - 8) \ll 8) \,|\, a.$$

If a character is at the end of a patterns, for example b in the pattern ab, then we assign the mask for

$$0 \ll 8 \,|\, b.$$

But now, the algorithms may *fail* to report the match of ab at bucket B_0 that ends at position 2 for the given text *babc*. The reason is that when scanning the second b, it looks and ors the mask for the super-character $(b[0 \ldots s - 8) \ll 8) \,|\, c$ instead of the mask for $0 \ll 8 \,|\, b$. The idea is to set the bit(s) that may introduce a wrong contradiction to 0 in the mask for the super-character beforehand. More precisely, if a character c appears at position 0 of a pattern (that is, a bit confusing, at the end!) in bucket B_j, then we set the bit at position j to 0 in the mask for the super-character

$$(x[0 \ldots s - 8) \ll 8) \,|\, c$$

for *every* character x. This can be done more efficiently, as in fact there are only 2^{s-8} such super-characters. The corrected compilation pseudocode is given in Algorithm 3. The changes or additions are highlighted in blue.

Algorithm 3 Compilation phase of the FDR string matcher

Assign each pattern to a bucket $B_1, \ldots, B_b$.
$M \leftarrow$ a dictionary mapping each character to a mask.
$s \leftarrow$ the number of bits in a super-character.
for $c = \underbrace{00\ldots00}_{s}$ to $\underbrace{11\ldots11}_{s}$ **do**
    $M[c] \leftarrow \underbrace{00\ldots00}_{64}\underbrace{11\ldots11}_{64}$ ▷ Initialize.
    for $j = 0$ to 7 and B_j is not empty **do**
        $\ell_j \leftarrow$ length of the longest pattern in bucket B_j.
        for $i = \ell_j$ to 7 **do**
            $M[c][8i + j] \leftarrow 0$ ▷ Set padding bits to 0.
        endfor
    endfor
endfor
for $j = 0$ to 7 **do**
    for each pattern x in bucket B_j **do**
        $f \leftarrow x[0]$ ▷ Handle the end character.
        for $y \leftarrow \underbrace{0\ldots0}_{s-8}$ to $\underbrace{1\ldots1}_{s-8}$ **do**
            $M[(y \ll 8)|f][8j] \leftarrow 0$
        endfor
        for $i = 1$ to length$(x) - 1$ **do**
            $M[x[i]][8i + j] \leftarrow 0$
        endfor
    endfor
endfor

We comment on the original implementation. In the execution phase, the original implementation lets us traverse j from 0 to 7 with skipping. This parameter is called *stride*. For example, if the stride is 2, then we traverse j from 0 to 7 with step 2. This is because each each step of j, we have to query the mask dictionary, which is not in constant time. The trade-off is that there are more positives to verify.

The related paper [1] claims that the algorithm outperforms Aho-Corasick (AC). But AC is already linear in the length of the text, asymptotically. So we may have to analyze very closely to the original implementation to understand the reason. In fact, there are some optimizations in the original implementation that are not reflected in the pseudocode. They leverage SIMD everywhere possible. For example, they encode the mask table in byte codes and access them using (aligned) addresses. Hence I would like to firstly understand why the complexity of FDR is not linear in the number of patterns. I begin at special cases to gain experience, then gradually study more general distributions of patterns and texts, and add super-characters, strides and hashing.

2 Analysis of a Special Cases

Let there be only one bucket and all patterns have the same length 8. This gives an intuition that the strength of the algorithms lies not mainly in parallelism but in the fact that false positives are rare. SIMD helps because there are long registers.

It is clear that the complexity of the compilation phase is $\mathcal{O}(p)$, since the number of characters to build masks are fixed.

In the execution phase, each position in the state mask R has to be or-ed at most 8 times, while 8 positions are updated at the same time. Each iteration scans 8 characters. Hence the complexity is roughly given by

$$\begin{aligned}
C &= \mathcal{O}\Big(\frac{n}{8}\Big) \times [\text{cost of each iteration}] \\
&= \mathcal{O}(n) \times \Big([\text{cost of ors}] \\
&\quad + \mathbb{P}(\text{no positive}) \times 0 \\
&\quad + \mathbb{P}(\text{positive but not matched}) \times [\text{cost of rejection}] \\
&\quad + \mathbb{P}(\text{positive and matched}) \times [\text{cost of confirmation}]\Big).
\end{aligned}$$

If hashing is used, then the cost of rejection would be less than the cost of confirmation. In our case, since we are doing exact string comparison, the cost of rejection and confirmation are in $\mathcal{O}(p)$.

Let $X_1, \ldots, X_p \overset{iid}{\sim} \mathcal{U}(\{0,1\}^{64})$ be the patterns, uniformly distributed in the set of binary strings of length 64. Let $Y \sim \mathcal{U}(\{0,1\}^8)$ be the text at an iteration. This implies that every character is drawn independently and uniformly at random. Suppose that there is only one bucket B_0 and all patterns are in this bucket. Let A be the event that there is a match ending at a position and B be the event that there is a positive. Then $A \subset B$. Let D be the event that all patterns are distinct. We rewrite the complexity as follows.

$$\begin{aligned}
C &= \mathcal{O}(n)(1 + \mathbb{P}(B^c \mid D) \times 0 + \mathcal{O}(p) \times (\mathbb{P}(A, B \mid D) + \mathbb{P}(A^c, B \mid D))) \\
&= \mathcal{O}(n)(1 + \mathcal{O}(p) \times (\mathbb{P}(A \mid D) + \mathbb{P}(B \mid D) - \mathbb{P}(A \mid D))) \\
&= \mathcal{O}(n)(1 + \mathcal{O}(p) \times \mathbb{P}(B \mid D)).
\end{aligned}$$

We check that $\mathbb{P}(B^c \mid D) + \mathbb{P}(A, B \mid D) + \mathbb{P}(A^c, B \mid D) = 1$, so the formulation should be correct. The algorithm is efficient when $\mathbb{P}(B \mid D)$ is negligible. In the first time, I tried to compute $\mathbb{P}(B \mid D)$ directly. Then I realized that it is not much different if we do not condition on D. Let me give this computation later. Maybe it can be used for more general cases. In fact, $\mathbb{P}(B \mid D)$ is close to $\mathbb{P}(B)$. We have

$$\begin{aligned}
\mathbb{P}(B \mid D) - \mathbb{P}(B) &= \frac{\mathbb{P}(B, D)}{\mathbb{P}(D)} - \mathbb{P}(B, D) - \mathbb{P}(B, D^c) \\
&\le \mathbb{P}(B, D)\left(\frac{1}{\mathbb{P}(D)} - 1\right) \\
&\le \mathbb{P}(D)\left(\frac{1}{\mathbb{P}(D)} - 1\right) \\
&= 1 - \mathbb{P}(D) = \mathbb{P}(D^c).
\end{aligned}$$

Now we bound $\mathbb{P}(D^c)$. This is the probability that there are two patterns that are the same. Let $N = 2^{64}$. Firstly, we choose 2 among p patterns and set them to be one of the N possible patterns. Then we choose the remaining $p - 2$ patterns from the remaining $N - 1$ possible patterns. There may be some over-counting, so we have

$$\mathbb{P}(D^c) \le \frac{\binom{p}{2} \times N \times N^{p-2}}{N^p} = \frac{\binom{p}{2}}{N}.$$

This is indeed negligible because N is very large. Now we compute $\mathbb{P}(B)$, the probability that there is a positive. Since Y is uniformly distributed and independent of the patterns, we can

fix $Y = 0^{64}$. Let us count the number of possible values of $X_1, \ldots, X_p$ such that there is a positive. There are in total N^p possible values of $X_1, \ldots, X_p$. For each position i from 0 to 7, there are $2^8 = 256$ possible characters. But 00000000 is forbidden. Hence there are 255 available possible characters allowed for the patterns. Therefore, there are 255^p possible values of $X_1[8i \ldots 8i{+}7], \ldots, X_p[8i \ldots 8i{+}7]$ such that there is no positive. Hence, there are $(N^p - 255^p)^8$ possible values of $X_1, \ldots, X_p$ such that there is a positive. Therefore,

$$\mathbb{P}(B) = \frac{(N^p - 255^p)^8}{N^{8p}} = \left(1 - \left(\frac{255}{256}\right)^p\right)^8.$$

When $p \geq 355$, $\mathbb{P}(B) \geq 0.1$. The algorithm starts to be inefficient. Suppose that p patterns are divided into 8 buckets uniformly. Then the new probability of a positive bounded by

$$1 - \left(1 - \left(1 - \left(\frac{255}{256}\right)^{p/8}\right)^8\right)^8.$$

This reaches 0.1 when $p \geq 1800$.

Let us use a different approach to compute $\mathbb{P}(B)$, which then can then be applied to compute $\mathbb{P}(B \mid D)$, and the case where super-characters are used. Again, since Y is uniformly distributed and independent of the patterns, we can fix $Y = 0^{64}$. Let $U = \{0,1\}^{64}$ be the universe. Define the sets

$$S_j = \{x \in U \mid x[8j \ldots 8j+7] = 0^8\}, j \in \{0, \ldots, 7\}.$$

Then B is the event that there is at least one pattern in each S_j i.e. the set $\{X_1, \ldots, X_p\}$ hits all S_j's. Then B^c is the event that there is some j such that $\{X_1, \ldots, X_p\}$ does not hit S_j.

$$B^c = \bigcup_{j=1}^{8} \underbrace{\bigcap_{i=1}^{p} \{X_i \notin S_j\}}_{E_j}.$$

By inclusion-exclusion,

$$\begin{aligned}
\mathbb{P}(B) &= 1 - \mathbb{P}\left(\bigcup_{j=1}^{8} E_j\right) \\
&= 1 - \sum_{\varnothing \neq L \subset \{0 \ldots 7\}} (-1)^{|L|+1} \mathbb{P}\left(\bigcap_{j \in L} E_j\right) \\
&= 1 + \sum_{\varnothing \neq L \subset \{0 \ldots 7\}} (-1)^{|L|} \mathbb{P}\left(\bigcap_{j=1}^{8} \bigcap_{i=1}^{p} \{X_i \notin S_j\}\right) \\
&= 1 + \sum_{\varnothing \neq L \subset \{0 \ldots 7\}} (-1)^{|L|} \mathbb{P}\left(\bigcap_{i=1}^{p} \left\{X_i \notin \bigcup_{j \in L} S_j\right\}\right) \\
&= 1 + \sum_{\varnothing \neq L \subset \{0 \ldots 7\}} (-1)^{|L|} \mathbb{P}\left(X_1 \notin \bigcup_{j \in L} S_j\right)^p \qquad \text{(since } X_i\text{'s are iid).}
\end{aligned}$$

Consider the event $V_L = (X_1 \notin \bigcup_{j \in L} S_j)$. To make this event happen, we can only choose $2^8 - 1 = 255$ possible strings for $X_1[8j \ldots 8j+7]$ for each $j \in L$ and $2^8 = 256$ possible strings for each $j \notin L$. Hence,

$$\mathbb{P}(V_L) = \frac{256^{8-|L|} \times 255^{|L|}}{N} = \left(\frac{255}{256}\right)^{|L|}.$$

There is $\binom{8}{\ell}$ subsets of $\{0 \dots 7\}$ of size ℓ. Hence,

$$\mathbb{P}(B) = 1 + \sum_{\ell=1}^{8} \binom{8}{\ell} (-1)^{\ell} \left(\frac{255}{256}\right)^{p\ell} = \left(1 - \left(\frac{255}{256}\right)^{p}\right)^{8}.$$

Note that in the computation of $\mathbb{P}(V_L)$, the numerator $256^{8-|L|} \times 255^{|L|} = N\,(255/256)^{|L|}$ is the number of patterns that can be chosen for each X_i. Conditioning on D means to choose p distinct patterns. Hence,

$$\mathbb{P}(\cap_{j \in L} E_j \mid D) = \frac{\binom{N(255/256)^{|L|}}{p}}{\binom{N}{p}} \approx \left(\frac{255}{256}\right)^{p|L|}.$$

Let us take super-characters into consideration. Recall that we may have to set the bit at position j to 0 in the mask for the super-character

$$(x[0 \dots s-7] \ll 8) \,|\, c$$

if c appears at the end of a pattern. That is, we only may have to set the bits in the *first* byte, which can be or-ed with the considered position of the text *once*. The other bits of the masks in the super-character case are the same as in the last case. Therefore, now we define

$$S_j^{(s)} = \{x \in U \mid x[j \dots j+7] = 0^8 \text{ and } x[j-8 \dots j+s-15] = 0^{s-8}\}, j \in \{1, \dots, 7\},$$

and similarly

$$E_j^{(s)} = \{X_1 \notin S_j^{(s)}, \dots, X_p \notin S_j^{(s)}\}, j \in \{1, \dots, 7\}.$$

That is, we do not care about position 0, and give an upper bound for $\mathbb{P}(B)$ by only considering the events $E_j^{(s)}$ for $j \in \{1, \dots, 7\}$. Let me first attempt like this, because handling which cases are some masks set to 0 at the first byte seems complicated to me at the moment, since we have to consider characters that appear both at the end and the middle of patterns. Similarly, we have

$$\begin{aligned}
\mathbb{P}(B) &\leq 1 - \mathbb{P}\left(\bigcup_{j=1}^{7} E_j^{(s)}\right) \\
&= 1 + \sum_{\varnothing \neq L \subset \{1,\dots,7\}} (-1)^{|L|} \mathbb{P}\left(X_1 \notin \bigcup_{j \in L} S_j^{(s)}\right)^p \\
&= 1 + \sum_{\varnothing \neq L \subset \{1,\dots,7\}} (-1)^{|L|} \mathbb{P}\left(V_L^{(s)}\right)^p
\end{aligned}$$

Now we compute $\mathbb{P}\left(V_L^{(s)}\right)$. Previously, we were able to "choose 255 among 256" because the S_j's forbid 0^8 at disjoint positions. Now the $S_j^{(s)}$'s forbid 0^8 at positions that may overlap. So we will decompose $\cup_{j \in L} S_j^{(s)}$ into $\cup_{C \in \mathcal{C}_L} (\cup_{j \in C} S_j)$, where each C is a *cluster* i.e. a maximal subset of

L containing consecutive elements. For example, if $L = \{1, 2, 4, 5, 6\}$, then there are two clusters $\{1, 2\}$ and $\{4, 5, 6\}$. Then

$$\mathbb{P}\left(V_L^{(s)}\right) = \prod_{C \in \mathcal{C}_L} \mathbb{P}\left(X_1 \notin \bigcup_{j \in C} S_j^{(s)}\right) = \prod_{C \in \mathcal{C}_L} \left(1 - \mathbb{P}\left(X_1 \in \bigcup_{j \in C} S_j^{(s)}\right)\right).$$

Let $R_j = [8j, 8j + 7] \cup [8j - 8, 8j + s - 15]$ be the positions corresponding to $S_j^{(s)}$. By inclusion-exclusion again.

$$P\left(X_1 \in \bigcup_{j \in C} S_j^{(s)}\right) = \sum_{\varnothing \neq K \subset C} (-1)^{|K|+1} \mathbb{P}\left(X_1 \in \bigcap_{j \in K} S_j^{(s)}\right).$$

The event $X_1 \in \bigcap\limits_{j \in K} S_j^{(s)}$ means that X_1 has 0's at the positions in $\bigcup_{j \in K} R_j$ and can be any string at the other positions. Hence,

$$P(V^{(s)}) = \sum_{\varnothing \neq K \subset L} (-1)^{|K|+1} \frac{1}{2^{|\bigcup_{j \in K} R_j|}}.$$

Now we have to compute $r_K = |\bigcup_{j \in K} R_j|$. We leverage *clusters* in K. Let $|K| = k \geq 1$. If K has u clusters, then

$$r_K = 8k + u(s - 8).$$

The problem is reduced to counting the number of partitions of a set of size k to have u clusters. There are two stages. The first stage is to choose the sizes of the clusters i.e. $x_1, \ldots, x_u \in \mathbb{N}^*$ such that

$$x_1 + \cdots + x_u = k.$$

By Euler's Candies Division, there are $\binom{k-1}{u-1}$ ways to do this. The second stage is to choose the gaps between the clusters i.e. $y_0, y_u \in \mathbb{N}$ and $y_1, \ldots, y_{u-1} \in \mathbb{N}^*$ such that

$$y_0 + x_1 + y_1 + \ldots + y_{u-1} + x_u + y_u = 7 \Leftrightarrow y_0 + \ldots + y_u = 7 - k.$$

Let $y_0' = y_0 + 1$ and $y_u' = y_u + 1$. Then $y_0', y_1, \ldots, y_{u-1}, y_u' \in \mathbb{N}^*$ and

$$y_0' + y_1 + \ldots + y_{u-1} + y_u' = 9 - k.$$

By Euler's Candies Division again, there are $\binom{8-k}{u}$ ways to do this. Note that $\binom{8-k}{u-1} = 0$ if $u > k$. Therefore, the number of ways to partition a subset of size k of $\{1, \ldots, 7\}$ such that there are u clusters is

$$\binom{k-1}{u-1}\binom{8-k}{u}.$$

From that formula, we derive that $u \in \min\{k, 8 - k\}$. Hence,

$$\mathbb{P}\left(X_1 \in \bigcap_{j \in K} S_j^{(s)}\right) = \sum_{k=1}^{7} \sum_{u=1}^{\min\{k,8-k\}} (-1)^k \binom{k-1}{u-1}\binom{8-k}{u} \frac{1}{2^{8k+u(s-8)}}.$$

I think we can elaborate more to count the number of subsets L having v_i clusters of size i for $i \in \{1, \ldots, 7\}$. I am not sure if I can get an asymptotic approximation to compare with the case without super-characters.

3 Problem Statement

In general, problem is stated as follows. Let $\mathcal{D}_P$ be the distribution of patterns and $\mathcal{D}_T$ be the distribution the text. Let the domain of the patterns be $\mathcal{X}$ and the domain of the text be $\mathcal{Y}$. We have to write the risk R_p i.e. the probability of a positive, as a function of $\mathcal{D}_P$ and $\mathcal{D}_T$ when p patterns are drawn from $\mathcal{D}_P$ and the text is drawn from $\mathcal{D}_T$.

An algorithm takes the patterns i.e. $\mathcal{X}^p$ and produces an assignment $f : \bigcup_{p \in \mathbb{N}} \mathcal{X}^p \to \{0, \ldots, 7\}$. It aims to minimize another risk over only $\mathcal{X}^p$.

Then we have to provide an upper bound for the difference between the two risks.

References

[1] Xiang Wang et al. "Hyperscan: A fast multi-pattern regex matcher for modern {CPUs}". In: *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 2019, pp. 631–648.